

Context-Aware Graph-Constrained Reasoning: Well-Formed Chains vs. Ontological Gates

Bernard Katamanso

August 2, 2026

1 What Does “Context-Aware” Mean?

In graph-constrained reasoning, the language model generates a reasoning path through a knowledge graph (KG). “Context awareness” refers to the use of information beyond the raw prompt to constrain generation. We identify five orthogonal dimensions:

- D1 Token-level context.** The trie encodes valid token sequences derived from KG edges. At every decode step, the `prefix_allowed_tokens_fn` callback restricts the model to tokens that appear in some valid path prefix. This is the tightest coupling between KG structure and generation.
- D2 Structural context.** The KG topology—edges, hops, connectivity—determines which paths exist. Depth-first search enumerates all paths up to L hops; the model can only follow edges that are present in the subgraph.
- D3 Semantic context.** The ontology (entity types, relation domain/range) determines which paths are *type-compatible*. A path may be topologically valid yet semantically incoherent (e.g. a currency where a language is expected). The TypeOracle’s `range_gate` and `type_gate` enforce semantic constraints.
- D4 Reasoning context.** Each generated hop narrows the set of candidate entities for the next hop. In iterative decoding, the head pool is updated after every hop, so the search space contracts as the model commits to specific entities.
- D5 Query context.** The question itself determines what answer types to look for. The regex-based `infer_answer_types` maps question words to expected Freebase type sets, which in turn activate the `type_gate` at the terminal hop.

These five layers are orthogonal. The DoG framework uses D1 (token) and D2 (structural). Our TypeOracle adds D3 (semantic) and D5 (query). D4 (reasoning context) remains underexploited: the *content* of generated tokens could further refine the search before the type oracle is consulted.

2 Well-Formed Chains vs. Ontological Gates

2.1 The Well-Formed Chain (DoG)

Following the DoG formulation [1], a *chain* through a KG $\mathcal{G} = (V, E)$ is defined as

$$c = (v_0, e_1, v_1, e_2, \dots, e_L, v_L),$$

where v_0 is the query entity, each $e_l \in E$ is a relation, and each $v_l \in V$ is an entity. The chain is **well-formed** if every hop follows a real edge:

$$\text{Well-formed}(c) \iff \forall l \in \{1, \dots, L\} : (v_{l-1}, e_l, v_l) \in E.$$

This is a purely *topological* constraint—it prevents hallucinated edges but says nothing about types.

2.2 The TypeOracle Gates

The TypeOracle adds two semantic gates on top of the topological constraint. Let $\mathcal{T} : V \rightarrow 2^{\text{Types}}$ denote the entity–type mapping (extracted from `common.topic.notable_types`) and let $\text{range} : \mathcal{E} \rightarrow 2^{\text{Types}}$ denote the relation range (hand-curated or auto-mined from the subgraph).

Gate 1: Range constraint.

$$\text{range_gate}(e_l, v_l) \iff \mathcal{T}(v_l) \cap \text{range}(e_l) \neq \emptyset \quad \vee \quad \mathcal{T}(v_l) = \emptyset \quad \vee \quad \text{range}(e_l) = \emptyset.$$

The disjunction with the empty-set cases encodes *conservatism*: if no type information is available, the edge is admitted.

Gate 2: Answer-type constraint (terminal hop).

$$\text{type_gate}(v_L, A, L, L) \iff \mathcal{T}(v_L) \cap A \neq \emptyset \quad \vee \quad \mathcal{T}(v_L) = \emptyset \quad \vee \quad A = \emptyset,$$

where $A \subseteq \text{Types}$ is the set of expected answer types inferred from the question. This gate fires only at the terminal hop ($l = L$).

Combined admission. A candidate edge (e_l, v_l) is admitted iff it passes both gates:

$$\text{admissible}(e_l, v_l) = \text{range_gate}(e_l, v_l) \wedge \text{type_gate}(v_l, A, l, L).$$

2.3 Formal Relationship

Theorem 1. *If every admitted edge satisfies the range gate, then every admitted chain is well-formed:*

$$\forall c : \text{admissible}(c) \implies \text{Well-formed}(c).$$

The converse does not hold.

Proof. Admissibility requires $\mathcal{T}(v_l) \cap \text{range}(e_l) \neq \emptyset$ for every hop. In particular, $\mathcal{T}(v_l) \neq \emptyset$ and $\text{range}(e_l) \neq \emptyset$, so the edge (v_{l-1}, e_l, v_l) must exist in the KG (otherwise $\text{range}(e_l)$ would not be defined). Hence the chain is well-formed.

For the converse, consider the edge `location.country.languages_spoken` with range `{Human Language}`. The entity `Jamaican dollar` has type `{Currency}`, so $\mathcal{T}(\text{Jamaican dollar}) \cap \{\text{Human Language}\} = \emptyset$. The edge exists in the KG (well-formed), but the range gate rejects it (not admissible). $\square$

2.4 Comparison

	Well-Formed Chain (DoG)	TypeOracle
Source	KG adjacency E	Ontology $\mathcal{T}$, range
Scope	All hops	All hops (range) + terminal (type)
Pruning power	Hallucinated edges only	Type-incompatible edges too
Complexity	$O(1)$ edge lookup	$O(1)$ set intersection
Catches	“ X doesn’t connect to Y ”	“ Y is a Currency, expected Language”

2.5 Illustrative Example

Question: “What language is spoken in Jamaica?”

Path: Jamaica $\rightarrow$ languages_spoken $\rightarrow$ Jamaican dollar

This path is *well-formed*: the edge exists in the KG. But it is *semantically wrong*: Jamaican dollar is a Currency, not a Language. The TypeOracle catches this because $\text{range}(\text{languages_spoken}) = \{\text{Human Language}\}$ and $\mathcal{T}(\text{Jamaican dollar}) = \{\text{Currency}\}$, so the intersection is empty.

3 Implications for Design

The well-formed chain is **necessary but not sufficient** for correct reasoning. The TypeOracle is a **semantic filter on top of a topological constraint**. Both layers are needed:

- **Topological (DoG):** prevents hallucinated structure.
- **Semantic (TypeOracle):** prevents type-incoherent structure.

Future work should exploit D4 (reasoning context) more aggressively: as the model generates each hop, the relation itself constrains the next entity—potentially without waiting for the type oracle.

References

- [1] Lee et al., “Decoding on Graph: Augmenting LLM Reasoning with Knowledge Graph Structured Search,” *ACL*, 2025. arXiv:2410.18415.